PRACTICAL REPORT
ON

ADVANCED DATA SCIENCE

SUBMITTED BY
AADITYAKUMAR MISHRA
ROLL NO: 17

SUBMITTED TO
MS. SOPHIA DCRUZ

MSc. (INFORMATION TECHNOLOGY) SEM - I

2026 – 2027



CONDUCTED AT
CHIKITSAK SAMUHA'S
S. S. & L.S. PATKAR COLLEGE OF ARTS & SCIENCE
AND
V. P. VARDE COLLEGE OF COMMERCE & ECONOMICS

An Empowered Autonomous college,
Affiliated to University of Mumbai
GOREGAON (W). MUMBAI -400062

CHIKITSAK SAMUHA'S

SIR SITARAM & LADY SHANTABAI PATKAR COLLEGE OF ARTS & SCIENCE

&

V.P. VARDE COLLEGE OF COMMERCE & ECONOMICS

GOREGAON (WEST), MUMBAI - 400 104.
An Empowered Autonomous College, University of Mumbai

CERTIFICATE

Certified that such of the experiments as have been duly signed were performed by Mr./Miss_____

Roll No. _____ of _____ class _____

Division_____ in the _____ Laboratory of this college during the year _____

Professor-in-Charge **Examiner** **Co-ordinator**

Date: _____ **_____Department**

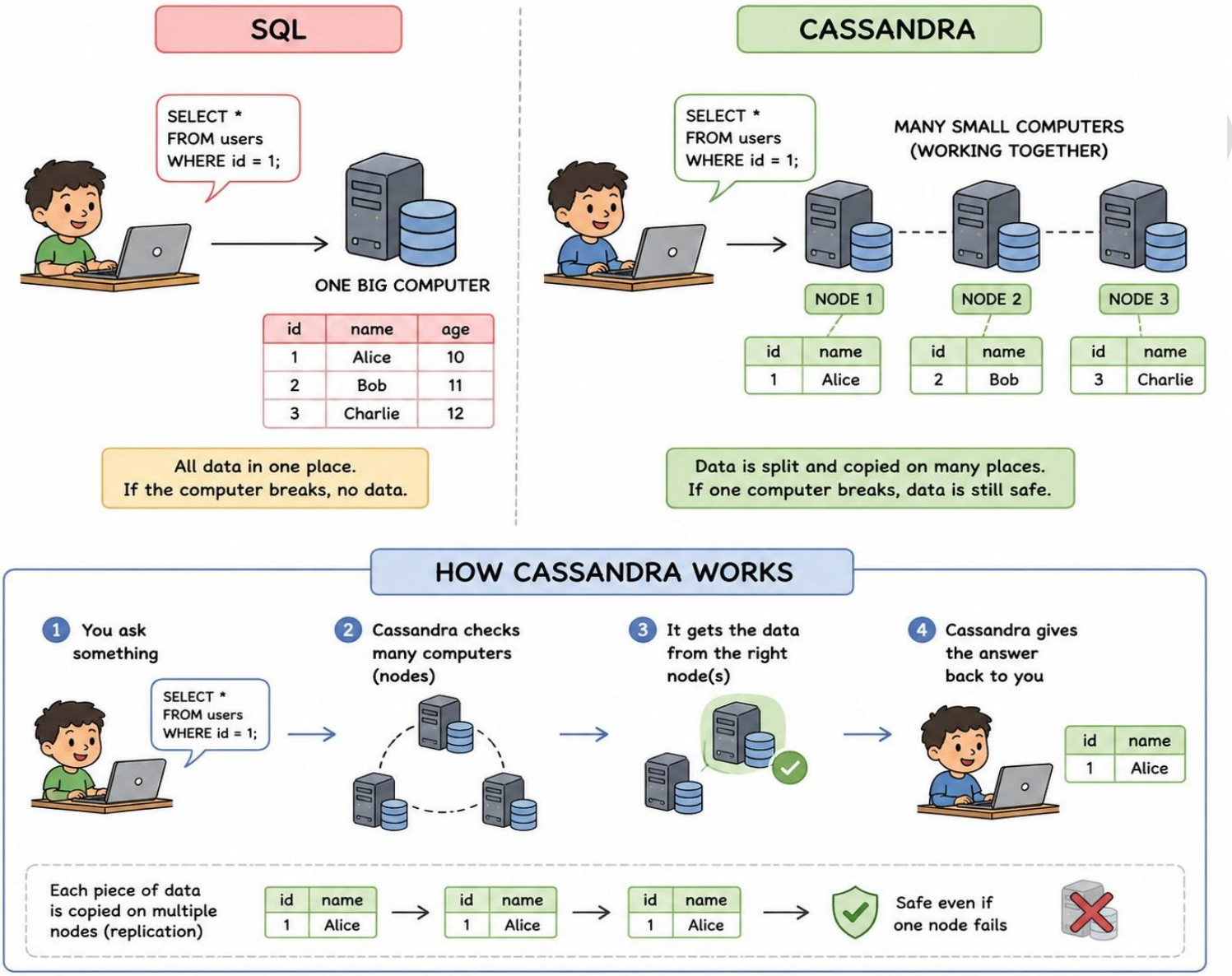
INDEX

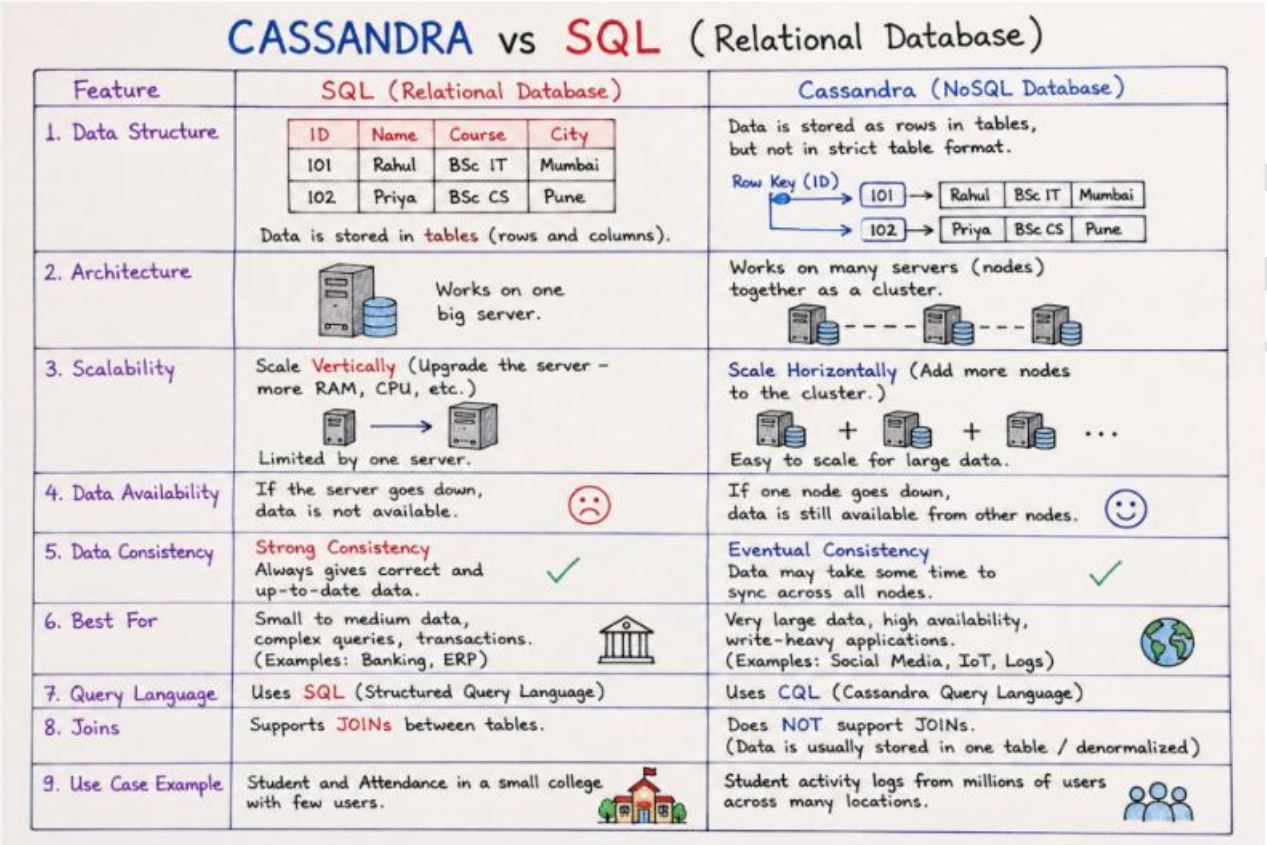
Practical No	Date	Practical Aim	Signature
1	8th August 2026	**Creating data model using Cassandra**	
2	22nd August 2026	**Conversion from different formats to HORUS format.**	
2A		**Text delimited CSV to HORUS**	
2B		**XML to HORUS**	
2C		**JSON to HORUS**	
2D		**Database to HORUS**	
2E		**Audio to HORUS**	
2F		**Video to HORUS**	
3	11th September 2026	**Utilities and Auditing**	
3A		**Fixer Utilities**	
3B		**Data Binning or Bucketing**	
3C		**Averaging of Data**	
3D		**Outlier Detection**	
3E		**Logging**	
4	19th September 2026	**Retrieving Data**	
4A		**Program to retrieve different attributes of data**	
4B1		**Designing a Routing Diagram for Vermeulen Company**	
4B2		**Planning a schedule of jobs to be performed for the router network.**	

4C	19th September 2026	**Picking Content for Billboards**	
4D1		**Planning the shipping rules for best-fit international logistics**	
4D2		**Adopt New Shipping Containers**	
4D3		**Creating a delivery route**	
4E		**Person Base data**	
5	28th September 2026	**Assessing Data**	
5A		**Drop the Columns Where All Elements Are Missing Values**	
5B		**Drop the Columns Where Any of the Elements Is Missing Values**	
5C		**Keep Only the Rows That Contain a Maximum of Two Missing Values**	
6	01st October 2026	**Processing Data -Build the time hub, links and satellites.**	
7	01st October 2026	**Transforming Data - To illustrate the consolidation process**	
8		**Organizing Data**	
8A		**Horizontal Style**	
8B		**Vertical Syle**	
8C		**Island Style**	
9		**Data Visualisation with PowerBI - Case Study on Sales Details**	

PRACTICAL 1

AIM: Creating data model using Cassandra





STEPS TO BE PERFORMED:

1. Download Cassandra

- Go to **https://cassandra.apache.org/**
- Download Cassandra 3.10.x for Windows.
- Extract the ZIP file, for example: **D:\apache-cassandra-3.10.2**

2. Install JDK 8

- Download and install JDK 8.
- Example installation folder: **C:\Program Files\Java\jdk1.8.0_202**

3. Set JAVA_HOME

- Open Environment Variables
- Search → Edit the system environment variables
- Advanced → Environment Variables
- Under System Variables, click New
- Variable name: **JAVA_HOME**
- Variable value: **C:\Program Files\Java\jdk1.8.0_202**

4. Add Java to PATH

- In System Variables, select **Path**
- Click Edit → New
- Add: **%JAVA_HOME%\bin or C:\Program Files\Java\jdk1.8.0_202\bin**
- On cmd check: **java -version**
- It should show Java 8.

5. Install Python 2.7

- Download Python 2.7.18
- Install it, for example: C:\Python27

Why Python 2.7?
Older Cassandra versions use an older cqlsh that requires Python 2.7.

6. Add Cassandra to PATH
- Go to System **Variables → Path → Edit → New**
- Add: **D:\apache-cassandra-3.11.2 \bin**

7. Start Cassandra
- Open Command Prompt, Type: **cd D:\apache-cassandra-3.11.2\bin**
- Now type: **cassandra**
- Wait for Cassandra to start.

8. Open CQL Shell
- Open another Command Prompt and type:**cd D:\apache-cassandra-3.11.2\bin**
- Next type: **C:\Python27\python.exe D:\apache-cassandra-3.11.2\bin\cqlsh.py**

If successful, you will see something similar to:

Connected to Test Cluster
[cqlsh 5.x.x | Cassandra 3.10.x]

COLLEGE ATTENDANCE MANAGEMENT SYSTEM

1. CREATE KEYSPACE
```
CREATE KEYSPACE college
WITH replication = {
   'class': 'SimpleStrategy',
   'replication_factor': 1
};
```

2. USE KEYSPACE
```
USE college;
```

3. CREATE STUDENTS TABLE
```
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name TEXT,
   course TEXT,
   semester INT,
   city TEXT,
   email TEXT
);
```

4. INSERT STUDENT DATA
```
INSERT INTO students
(student_id, name, course, semester, city, email)
VALUES (101, 'Rahul Sharma', 'BSc IT', 5, 'Mumbai', 'rahul@gmail.com');

INSERT INTO students
(student_id, name, course, semester, city, email)
VALUES (102, 'Priya Patil', 'BSc IT', 5, 'Pune', 'priya@gmail.com');

INSERT INTO students
(student_id, name, course, semester, city, email)
VALUES (103, 'Amit Shah', 'BSc CS', 5, 'Mumbai', 'amit@gmail.com');

INSERT INTO students
(student_id, name, course, semester, city, email)
```

Aaditya - ADS

VALUES (104, 'Sneha Joshi', 'BSc IT', 6, 'Nashik', 'sneha@gmail.com');

5. DISPLAY ALL STUDENTS
SELECT * FROM students;

studentid	city	course	email	name	semester
102	Mumbai	BSC-IT	raj@gmail.com	Raj	3
101	Mumbai	BSC-IT	rahul@gmail.com	Rahul Sharma	5
103	Mumbai	BSC-IT	aditya@gmail.com	aditya	2

6. FIND A PARTICULAR STUDENT
SELECT * FROM students WHERE student_id = 101;

studentid	city	course	email	name	semester
101	Mumbai	BSC-IT	rahul@gmail.com	Rahul Sharma	5

7. UPDATE STUDENT INFORMATION
UPDATE students
SET city = 'Thane'
WHERE student_id = 101;

8. VERIFY UPDATED STUDENT
SELECT * FROM students WHERE student_id = 101;

studentid	city	course	email	name	semester
101	Thane	BSC-IT	rahul@gmail.com	Rahul Sharma	5

9. CREATE STUDENT ATTENDANCE TABLE
```
CREATE TABLE student_attendance (
   student_id INT,
   attendance_date DATE,
   subject TEXT,
   status TEXT,
   PRIMARY KEY (student_id, attendance_date, subject)
);
```

10. INSERT ATTENDANCE DATA
```
INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (101, '2026-08-01', 'Artificial Intelligence', 'Present');

INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (101, '2026-08-01', 'Cyber Security', 'Present');

INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (101, '2026-08-02', 'Artificial Intelligence', 'Absent');

INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (101, '2026-08-02', 'Cyber Security', 'Present');

INSERT INTO student_attendance
```

(student_id, attendance_date, subject, status)
VALUES (102, '2026-08-01', 'Artificial Intelligence', 'Present');

INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (102, '2026-08-01', 'Cyber Security', 'Absent');

INSERT INTO student_attendance
(student_id, attendance_date, subject, status)
VALUES (102, '2026-08-02', 'Artificial Intelligence', 'Present');

11. DISPLAY ALL ATTENDANCE
SELECT * FROM student_attendance;

```
 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+---------
       102 |     2026-08-01 | Artificial Intelligence | Present
       102 |     2026-08-01 |          Cyber Security |  Absent
       102 |     2026-08-02 | Artificial Intelligence | Present
       101 |     2026-08-01 | Artificial Intelligence | Present
       101 |     2026-08-01 |          Cyber Security | Present
       101 |     2026-08-02 | Artificial Intelligence |  Absent
       101 |     2026-08-02 |          Cyber Security | Present
```

12. DISPLAY ATTENDANCE OF STUDENT 101
SELECT * FROM student_attendance WHERE student_id = 101;

```
cqlsh:college> select * from studentAttendance where studentId=101;

 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+---------
       101 |     2026-08-01 | Artificial Intelligence | Present
       101 |     2026-08-01 |          Cyber Security | Present
       101 |     2026-08-02 | Artificial Intelligence |  Absent
       101 |     2026-08-02 |          Cyber Security | Present
```

13. DISPLAY ATTENDANCE OF STUDENT 101
FOR A PARTICULAR DATE
SELECT * FROM student_attendance WHERE student_id = 101
AND attendance_date = '2026-08-01';

```
 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+---------
       101 |     2026-08-01 | Artificial Intelligence | Present
       101 |     2026-08-01 |          Cyber Security | Present
```

14. DISPLAY ATTENDANCE OF STUDENT 101
FOR ARTIFICIAL INTELLIGENCE
SELECT * FROM student_attendance WHERE student_id = 101
AND attendance_date = '2026-08-01'
AND subject = 'Artificial Intelligence';

```
 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+---------
       101 |     2026-08-01 | Artificial Intelligence | Present
```

15. UPDATE ATTENDANCE
UPDATE student_attendance
SET status = 'Absent'
WHERE student_id = 101

```
AND attendance_date = '2026-08-01'
AND subject = 'Artificial Intelligence';
```

16. VERIFY UPDATED ATTENDANCE

```
SELECT *
FROM student_attendance
WHERE student_id = 101
AND attendance_date = '2026-08-01'
AND subject = 'Artificial Intelligence';
```

```
 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+--------
       101 |     2026-08-01 | Artificial Intelligence | Absent
```

17. DELETE AN ATTENDANCE RECORD

```
DELETE FROM student_attendance
WHERE student_id = 102
AND attendance_date = '2026-08-02'
AND subject = 'Artificial Intelligence';
```

18. VERIFY DELETED RECORD

```
SELECT * FROM student_attendance WHERE student_id = 102;
```

```
 studentid | attendancedate | subject                | status
-----------+----------------+------------------------+---------
       102 |     2026-08-01 | Artificial Intelligence | Present
       102 |     2026-08-01 |          Cyber Security |  Absent
```

19. CREATE A SECOND DATA MODEL FOR QUERYING ATTENDANCE BY SUBJECT

```
CREATE TABLE attendance_by_subject (
   subject TEXT,
   attendance_date DATE,
   student_id INT,
   student_name TEXT,
   status TEXT,
   PRIMARY KEY ((subject, attendance_date), student_id)
);
```

20. INSERT DATA INTO SECOND TABLE

```
INSERT INTO attendance_by_subject
(subject, attendance_date, student_id, student_name, status)
VALUES ('Artificial Intelligence', '2026-08-01', 101, 'Rahul Sharma', 'Present');

INSERT INTO attendance_by_subject
(subject, attendance_date, student_id, student_name, status)
VALUES ('Artificial Intelligence', '2026-08-01',102, 'Priya Patil', 'Present');

INSERT INTO attendance_by_subject
(subject, attendance_date, student_id, student_name, status)
VALUES ('Artificial Intelligence', '2026-08-01',103, 'Amit Shah', 'Absent');

INSERT INTO attendance_by_subject
(subject, attendance_date, student_id, student_name, status)
VALUES('Cyber Security', '2026-08-01',101, 'Rahul Sharma', 'Present');

INSERT INTO attendance_by_subject
(subject, attendance_date, student_id, student_name, status)
```


Aditya ADS

VALUES ('Cyber Security', '2026-08-01',102, 'Priya Patil', 'Absent');

21. DISPLAY ALL SUBJECT-WISE ATTENDANCE

SELECT * FROM attendance_by_subject;

subject	attendance_date	student_id	status	student_name
Artificial Intelligence	2026-08-01	101	Present	Rahul Sharma
Artificial Intelligence	2026-08-01	102	Present	Priya Patil
Artificial Intelligence	2026-08-01	103	Absent	Amit Shah
Cyber Security	2026-08-01	101	Present	Rahul Sharma
Cyber Security	2026-08-01	102	Absent	Priya Patil

22. FIND ALL STUDENTS WHO ATTENDED AI ON 1st AUGUST 2026

SELECT * FROM attendance_by_subject
WHERE subject = 'Artificial Intelligence'
AND attendance_date = '2026-08-01';

subject	attendance_date	student_id	status	student_name
Artificial Intelligence	2026-08-01	101	Present	Rahul Sharma
Artificial Intelligence	2026-08-01	102	Present	Priya Patil
Artificial Intelligence	2026-08-01	103	Absent	Amit Shah

23. UPDATE SUBJECT-WISE ATTENDANCE

UPDATE attendance_by_subject
SET status = 'Absent'
WHERE subject = 'Artificial Intelligence'
AND attendance_date = '2026-08-01'
AND student_id = 101;

24. VERIFY UPDATE

SELECT * FROM attendance_by_subject
WHERE subject = 'Artificial Intelligence'
AND attendance_date = '2026-08-01'
AND student_id = 101;

subject	attendance_date	student_id	status	student_name
Artificial Intelligence	2026-08-01	101	Absent	Rahul Sharma

25. DELETE SUBJECT-WISE ATTENDANCE

DELETE FROM attendance_by_subject
WHERE subject = 'Cyber Security'
AND attendance_date = '2026-08-01'
AND student_id = 102;

26. DISPLAY FINAL DATA

SELECT * FROM students;
SELECT * FROM student_attendance;
SELECT * FROM attendance_by_subject;

subject	attendance_date	student_id	status	student_name
Artificial Intelligence	2026-08-01	101	Absent	Rahul Sharma
Artificial Intelligence	2026-08-01	102	Present	Priya Patil
Artificial Intelligence	2026-08-01	103	Absent	Amit Shah
Cyber Security	2026-08-01	101	Present	Rahul Sharma

PRACTICAL 2

AIM: Conversion from different formats to HORUS format

THEORY:
HORUS can be understood in this practical as a common/standard format used to represent different types of data.

2A - CSV to HORUS

THEORY:
CSV stands for Comma-Separated Values. It stores data in the form of rows and columns.

CODE:

```
import pandas as pd
sInputFileName='D:/practical-data-science/VKHCG/05-DS/9999-Data/Country_Code.csv'
InputData=pd.read_csv(sInputFileName,encoding="latin-1")
print('Input Data Values ===================================')
print(InputData)
print('=====================================================')
ProcessData=InputData
ProcessData.drop('ISO-2-CODE', axis=1,inplace=True)
ProcessData.drop('ISO-3-Code', axis=1,inplace=True)
ProcessData.rename(columns={'Country': 'CountryName'}, inplace=True)
ProcessData.rename(columns={'ISO-M49': 'CountryNumber'}, inplace=True)
ProcessData.set_index('CountryNumber', inplace=True)
ProcessData.sort_values('CountryName', axis=0, ascending=False, inplace=True)
print('Process Data Values ================================')
print(ProcessData)
print('=====================================================')
OutputData=ProcessData
sOutputFileName='D:/practical-data-science/VKHCG/05-DS/9999-Data/HORUS-CSV-Country.csv'
OutputData.to_csv(sOutputFileName, index = True)
print('CSV to HORUS - Done')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2>python prac2a.py
Input Data Values ===================================
         Country ISO-2-CODE ISO-3-Code  ISO-M49
0          India         IN        IND      356
1  United States         US        USA      840
2  United Kingdom        GB        GBR      826
3        Germany         DE        DEU      276
=====================================================
Process Data Values ================================
                CountryName
CountryNumber
840           United States
826          United Kingdom
356                   India
276                 Germany
=====================================================
CSV to HORUS - Done
```

2B - XML to HORUS

THEORY:

- ➢ XML stands for eXtensible Markup Language.
- ➢ It stores data using tags.

CODE:

```
import pandas as pd
import xml.etree.ElementTree as ET
def xml2df(xml_data):
    root = ET.fromstring(xml_data)
    records = []
    for child in root:
        record = {}
        for x in child:
            record[x.tag] = x.text
        records.append(record)
    return pd.DataFrame(records)
input_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/Country_Code.xml'
output_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/HORUS-XML-Country.csv'
xml_data = open(input_file).read()
data = xml2df(xml_data)
data.drop(['ISO-2-CODE', 'ISO-3-Code'], axis=1, inplace=True)
data.rename(columns={
    'Country': 'CountryName',
    'ISO-M49': 'CountryNumber'
}, inplace=True)

data.set_index('CountryNumber', inplace=True)
data.sort_values('CountryName', ascending=False, inplace=True)
print(data)
data.to_csv(output_file, index=False)
print("XML to HORUS - Done")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2>python prac2b.py
                CountryName
CountryNumber
840            United States
826           United Kingdom
356                    India
276                  Germany
XML to HORUS - Done
```

2C - JSON to HORUS

THEORY:

- ➢ JSON stands for JavaScript Object Notation.
- ➢ It represents data using key-value pairs.

CODE:

```
import pandas as pd
input_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/Country_Code.json'
output_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/HORUS-JSON-Country.csv'
data = pd.read_json(input_file, orient='index', encoding='latin-1')
print("Input Data Values")
print(data)
data.drop(['ISO-2-CODE', 'ISO-3-Code'], axis=1, inplace=True)
data.rename(columns={
    'Country': 'CountryName',
    'ISO-M49': 'CountryNumber'
}, inplace=True)
data.set_index('CountryNumber', inplace=True)
data.sort_values('CountryName', ascending=False, inplace=True)
print("Process Data Values")
print(data)
data.to_csv(output_file, index=True)
print("JSON to HORUS - Done")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2>python prac2c.py
Input Data Values

          Country ISO-2-CODE ISO-3-Code  ISO-M49
0           India         IN        IND      356
1   United States         US        USA      840
2  United Kingdom         GB        GBR      826
3         Germany         DE        DEU      276
============================================================
Process Data Values

                CountryName
CountryNumber
840             United States
826            United Kingdom
356                     India
276                   Germany
JSON to HORUS - Done
```

2D - Database to HORUS

THEORY:

- ➢ MySQL is a relational database management system (RDBMS).
- ➢ Data is stored in tables containing rows and columns.
- ➢ The program connects to the MySQL database, retrieves the required records and converts them into the HORUS format.

CODE:

```
import pandas as pd
import sqlite3 as sq
input_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/utility.db'
table = 'Country_Code'
output_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/HORUS-CSV-db_Country.csv'
conn = sq.connect(input_file)
data = pd.read_sql_query('SELECT * FROM ' + table, conn)
print("Input Data Values")
print(data)
data.drop(['ISO-2-CODE', 'ISO-3-Code'], axis=1, inplace=True)
data.rename(columns={
    'Country': 'CountryName',
    'ISO-M49': 'CountryNumber'
}, inplace=True)
data.set_index('CountryNumber', inplace=True)
data.sort_values('CountryName', ascending=False, inplace=True)
print("Process Data Values")
print(data)
data.to_csv(output_file, index=True)
conn.close()
print("Database to HORUS - Done")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2>python prac2d.py
Input Data Values
          Country ISO-2-CODE ISO-3-Code  ISO-M49
0           India         IN        IND      356
1   United States         US        USA      840
2  United Kingdom         GB        GBR      826
3         Germany         DE        DEU      276

Process Data Values
                CountryName
CountryNumber
840             United States
826            United Kingdom
356                     India
276                   Germany

Database to HORUS - Done
```

2E - AUDIO to HORUS

THEORY:

- Audio contains sound information represented digitally.
- The audio file can be read and processed so that its information can be represented in the required HORUS format.
- The practical PDF includes **AUDIO to HORUS** as the final conversion task.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.io import wavfile

def show_info(name, data, rate):
    print("\n--- Audio Info ---")
    print("Audio:", name)
    print("Rate:", rate)
    print("Shape:", data.shape)
    print("Min, Max:", data.min(), data.max())

    # Plot the channels visually
    plt.figure()
    for c in range(data.shape[1]):
        plt.plot(data[:, c], label='Ch' + str(c + 1))
    plt.legend()
    plt.title(f"Audio Waveform - {name}")
    plt.show()

# Local folder path pointing to your active workspace folder
path = "C:/Users/ADITYA/Downloads/Ads_Practical/Practical2/Prac2e/"

# 2 Channel Audio Processing
rate, data = wavfile.read(path + '2ch-sound.wav')
show_info("2 channel", data, rate)
df = pd.DataFrame(data)
df.columns = ['Ch1', 'Ch2']
df.to_csv(path + 'HORUS-Audio-2ch.csv', index=False)

# 4 Channel Audio Processing
rate, data = wavfile.read(path + '4ch-sound.wav')
show_info("4 channel", data, rate)
df = pd.DataFrame(data)
df.columns = ['Ch1', 'Ch2', 'Ch3', 'Ch4']
df.to_csv(path + 'HORUS-Audio-4ch.csv', index=False)

# 6 Channel Audio Processing
rate, data = wavfile.read(path + '6ch-sound.wav')
show_info("6 channel", data, rate)
df = pd.DataFrame(data)
df.columns = ['Ch1', 'Ch2', 'Ch3', 'Ch4', 'Ch5', 'Ch6']
df.to_csv(path + 'HORUS-Audio-6ch.csv', index=False)

# 8 Channel Audio Processing
rate, data = wavfile.read(path + '8ch-sound.wav')
show_info("8 channel", data, rate)
df = pd.DataFrame(data)
df.columns = ['Ch1', 'Ch2', 'Ch3', 'Ch4', 'Ch5', 'Ch6', 'Ch7', 'Ch8']
df.to_csv(path + 'HORUS-Audio-8ch.csv', index=False)

print("\nAudio to HORUS - Done")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2\Prac2e>python prac2e.py

--- Audio Info ---
Audio: 2 channel
Rate: 11025
Shape: (5512, 2)
Min, Max: -999 999

--- Audio Info ---
Audio: 4 channel
Rate: 11025
Shape: (5512, 4)
Min, Max: -999 999

--- Audio Info ---
Audio: 6 channel
Rate: 11025
Shape: (5512, 6)
Min, Max: -999 999

--- Audio Info ---
Audio: 8 channel
Rate: 11025
Shape: (5512, 8)
Min, Max: -999 999

Audio to HORUS - Done
```

2F - VIDEO to HORUS

THEORY:

- ➢ A video consists of a sequence of frames, usually combined with audio.
- ➢ The video can be read and processed frame-by-frame or as multimedia data and represented in the required HORUS format.
- ➢ The uploaded practical specifically includes VIDEO to HORUS.

CODE:

```
import os
import shutil
import cv2
input_file = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/dog.mp4'
output_dir = 'D:/practical-data-science/VKHCG/05-DS/9999-Data/temp'
if os.path.exists(output_dir):
    shutil.rmtree(output_dir)
os.makedirs(output_dir)
print("Start Movie to Frames")
video = cv2.VideoCapture(input_file)
success, image = video.read()
count = 0
while success:
    frame = output_dir + '/dog-frame-' + format(count, '04d') + '.jpg'
    cv2.imwrite(frame, image)
    print("Extracted:", frame)
    if count >= 100:
        break
    success, image = video.read()
    count += 1
video.release()
print("Generated:", count + 1, "Frames")
print("Movie to Frames HORUS - Done")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical2>python prac2f.py
Start Movie to Frames
Extracted: Prac2f/dog-frame-0000.jpg
Extracted: Prac2f/dog-frame-0001.jpg
Extracted: Prac2f/dog-frame-0002.jpg
Extracted: Prac2f/dog-frame-0003.jpg
Extracted: Prac2f/dog-frame-0004.jpg
Extracted: Prac2f/dog-frame-0005.jpg
Extracted: Prac2f/dog-frame-0006.jpg
Extracted: Prac2f/dog-frame-0007.jpg
Extracted: Prac2f/dog-frame-0008.jpg
Extracted: Prac2f/dog-frame-0009.jpg
Extracted: Prac2f/dog-frame-0010.jpg
 Generated: 101 Frames
 Movie to Frames HORUS - Done
```

Aaditya - ADS

PRACTICAL 3

AIM: Utilities and Auditing

3A - Fixers Utilities

THEORY:

- Data collected from different sources may contain unwanted spaces, special characters, or inconsistent formats.
- Fixer utilities are used to clean and standardize data before further processing.
- strip() removes unnecessary spaces from the beginning and end of a string.
- The program checks every character using string.printable and keeps only printable characters.
- strftime() converts a date into the required format.

CODE:

```
import string
import datetime as dt
# 1 Removing leading or lagging spaces from a data entry
print('1. Removing leading or lagging spaces from a data entry: \n');
baddata =" Data Science with too many spaces is bad!!! "
print('Raw Data >',baddata,'<')
cleandata=baddata.strip()
print('Clean Data >',cleandata,'<')
# 2 Removing nonprintable characters from a data entry
print('\n2. Removing nonprintable characters from a data entry:\n')
baddata = "Data\x00Science with\x02 funny characters is \x10bad!!!"
cleandata = ""
for ch in baddata:
    if ch in string.printable:
        cleandata = cleandata + ch
print("Bad Data :", baddata)
print("Clean Data :", cleandata)
# 3 Reformatting data entry to match specific formatting criteria.
# Convert YYYY/MM/DD to DD Month YYYY
print('\n3. Reformatting data entry:\n')
baddate = dt.date(2019, 10, 31)
baddata = baddate.strftime('%Y-%m-%d')
gooddata = baddate.strftime('%d %B %Y')
print('Bad Data :', baddata)
print('Good Data:', gooddata)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical3>python Prac3a.py
1. Removing leading or lagging spaces from a data entry:

Raw Data >  Data Science with too many spaces is bad!!!  <
Clean Data > Data Science with too many spaces is bad!!! <

2. Removing nonprintable characters from a data entry:

Bad Data : DataScience with funny characters is bad!!!
Clean Data : DataScience with funny characters is bad!!!

3. Reformatting data entry:

Bad Data : 2019-10-31
Good Data: 31 October 2019
```

Aditya ADS

3B - Data Binning or Bucketing

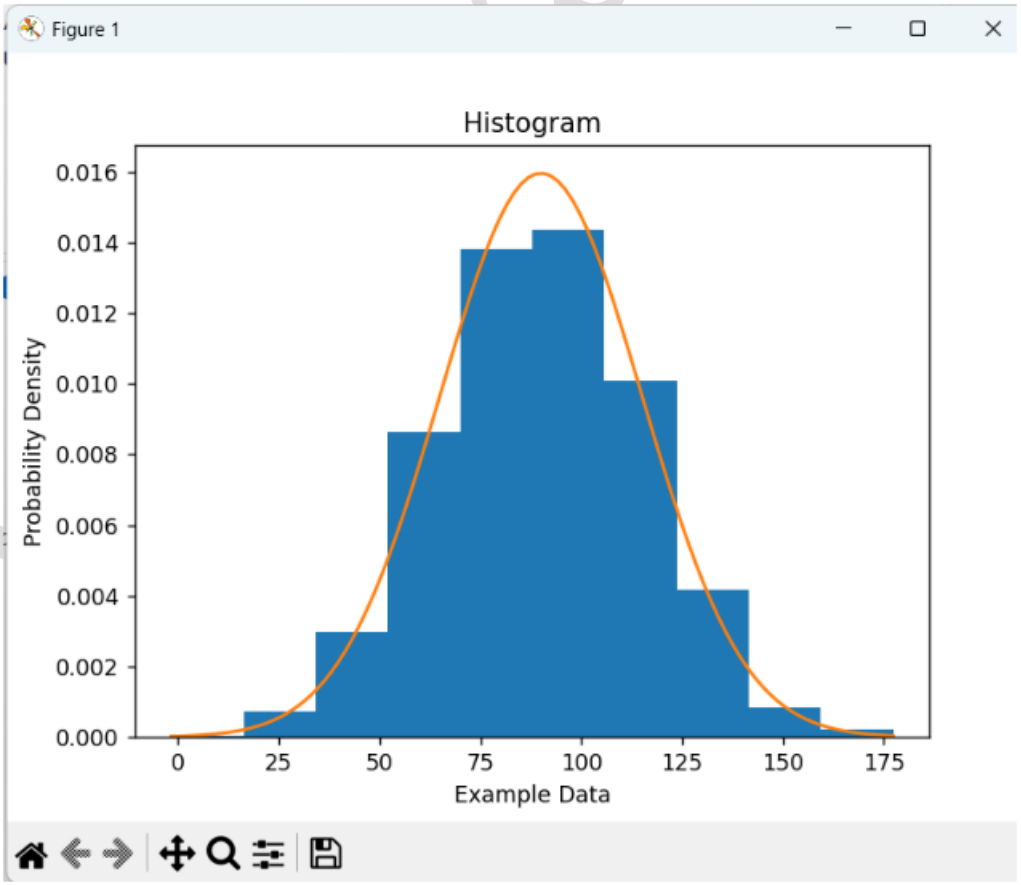
THEORY:

- ➢ Data binning means dividing continuous numerical data into groups or intervals called bins. A histogram is used to show how the values are distributed.
- ➢ In the code, random data is generated using a normal distribution and then displayed using a histogram.
- ➢ np.random.normal() → generates normally distributed data.
- ➢ plt.hist() → creates the histogram.
- ➢ bins=10 → divides data into 10 groups.
- ➢ norm.pdf() → generates the normal distribution curve.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
# Generate data
data = np.random.normal(90, 25, 5000) #(mean, sd, no. of samples)
# Create histogram
plt.hist(data, bins=10, density=True)
# Normal distribution curve
x = np.linspace(min(data), max(data), 100)
plt.plot(x, norm.pdf(x, 90, 25))
plt.xlabel("Example Data")
plt.ylabel("Probability Density")
plt.title("Histogram")
plt.show()
```

OUTPUT:



3C - Averaging of data

THEORY:

- Averaging is used to find the **mean value** of numerical data.
- The program reads selected columns from a CSV file and calculates the average latitude for each country and place

CODE:

```
import pandas as pd
data = pd.read_csv(
   'D:/practical-data-science/VKHCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv',
   usecols=['Country', 'Place Name', 'Latitude', 'Longitude'],
   encoding='latin-1'
)
result = data.groupby(['Country', 'Place Name'])['Latitude'].mean()
print(result)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical3>python Prac3c.py
Country        Place Name
India          Delhi          28.61695
               Mumbai         19.07900
United States  Los Angeles    34.05610
               New York       40.71640
Name: Latitude, dtype: float64
```

3D - Outlier Detection

THEORY:

- An outlier is a value that is **significantly different from the other values** in a dataset.
- Here, the program selects London records, calculates the mean and standard deviation of latitude, and identifies values outside the range:
- **Lower limit = Mean − Standard Deviation**
- **Upper limit = Mean + Standard Deviation**

CODE:

```
import pandas as pd
data = pd.read_csv(
   'D:/practical-data-science/VKHCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv',
   usecols=['Place Name', 'Latitude'],
   encoding='latin-1'
)
london = data[data['Place Name'] == 'London']
mean = london['Latitude'].mean()
std = london['Latitude'].std()
upper = mean + std
lower = mean - std
print("Upper Outliers:")
print(london[london['Latitude'] > upper])
print("Lower Outliers:")
print(london[london['Latitude'] < lower])
print("Not Outliers:")
print(london[(london['Latitude'] >= lower) &
        (london['Latitude'] <= upper)])
```


Aditya - ADS

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical3>python Prac3d.py
Upper Outliers:
  Place Name  Latitude
4     London      89.9
Lower Outliers:
  Place Name  Latitude
5     London      12.1
Not Outliers:
  Place Name  Latitude
0     London   51.5074
1     London   51.5200
2     London   51.4900
3     London   51.5150
```

3E - Logging

THEORY:

- ➢ Logging is used to record information about what happens while a program is running. It is useful for debugging, monitoring, and identifying errors.
- ➢ The program creates different types of log messages such as DEBUG, INFO, WARNING and ERROR.
- ➢ DEBUG: Detailed information for debugging
- ➢ INFo: General information
- ➢ WARNING: Something unusual may have happened
- ➢ ERROR: An error has occurred

CODE:

```
import logging
logging.basicConfig(
   level=logging.DEBUG,
   format='%(levelname)s - %(message)s',
   handlers=[
      logging.FileHandler('application.log'),
      logging.StreamHandler()
   ]
)
logging.debug("Debug message")
logging.info("Information message")
logging.warning("Warning message")
logging.error("Error message")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical3>python Prac3e.py
DEBUG - Debug message
INFO - Information message
WARNING - Warning message
ERROR - Error message
```

Aditya - ADS

PRACTICAL 4

AIM: Retrieving Data

4A - Program to retrieve different attributes of data

THEORY:

- Data retrieval means reading and extracting required information from data sources. Pandas is mainly used to read, clean, modify and save CSV data.
- **The program:**
  - Reads a CSV file.
  - Displays number of rows and columns.
  - Displays column names.
  - Removes unnecessary spaces from column names.
  - Replaces spaces with dots.
  - Adds a row identifier.
  - Displays the first five records.
  - Saves the cleaned dataset.
- **Important functions:**
  - **pd.read_csv()** → reads CSV.
  - **data.shape** → gives rows and columns.
  - **data.columns** → gives column names.
  - **str.strip()** → removes spaces.
  - **str.replace()** → replaces characters.
  - **head()** → displays first records.
  - **to_csv()** → saves data.

CODE:

```
import pandas as pd

# File path
file_path = "D:/practical-data-science/VKHCG/01-Vermeulen/00-RawData/IP_DATA_ALL.csv"

# Read CSV file
data = pd.read_csv(file_path, encoding="latin-1")

# Display number of rows and columns
print("Rows:", data.shape[0])
print("Columns:", data.shape[1])

# Display column names
print("\n### Raw Data ###")
for column in data.columns:
    print(column)

# Fix column names
print("\n### Fixed Data ###")

data.columns = data.columns.str.strip()
data.columns = data.columns.str.replace(" ", ".")

# Display fixed column names
for column in data.columns:
    print(column)
# Add Row ID
data.index.name = "RowID"

# Display first 5 rows
print("\n### Data with ID ###")
```


Aaditya - ADS

```
print(data.head())

# Save the fixed dataset
output_file =
"D:/practical-data-science/VKHCG/01-Vermeulen/01-Retrieve/01-EDS/02-Python/Retrieve_IP_DAT
A.csv"

data.to_csv(output_file, encoding="latin-1")

print("\nDone!")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4a.py
Rows: 5
Columns: 5

### Raw Data ###
Country
Place Name
Latitude
Longitude
IP Address

### Fixed Data ###
Country
Place.Name
Latitude
Longitude
IP.Address

### Data with ID ###
              Country   Place.Name  Latitude  Longitude   IP.Address
RowID
0               India       Mumbai   19.0760    72.8777  192.168.1.1
1               India        Delhi   28.6139    77.2090  192.168.1.2
2       United States     New York   40.7128   -74.0060  172.16.0.45
3       United States  Los Angeles   34.0522  -118.2437  172.16.0.46
4      United Kingdom       London   51.5074    -0.1278    10.0.0.15

Done!
```

4B1 Designing a Routing Diagram for Vermeulen Company

THEORY:

- This program calculates the distance between different locations using their latitude and longitude. It uses the Haversine formula, which calculates approximate distance between two points on Earth.
- Main steps
  - Read location data.
  - Remove duplicates.
  - Create all possible location combinations.
  - Calculate distance between locations.
  - Store distance in kilometres.
  - Convert kilometres into miles.
  - Save the result.
- Latitude: It represents the north-south position of a location.
- Longitude: It represents the east-west position.
- Haversine formula: Calculating distance between two geographical coordinates.
- Cross Join: It creates combinations between every row of one dataset and every row of another dataset.

CODE:

```
import pandas as pd
from math import radians, cos, sin, asin, sqrt

# Function to calculate distance
def haversine(lon1, lat1, lon2, lat2):
    lon1, lat1, lon2, lat2 = map(radians, [lon1, lat1, lon2, lat2])

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = sin(dlat / 2) ** 2 + cos(lat1) * cos(lat2) * sin(dlon / 2) ** 2
    c = 2 * asin(sqrt(a))

    return round(6371 * c, 3)   # Distance in km


# Read the CSV file
file_path = "D:/practical-data-science/VKHCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv"

data = pd.read_csv(
    file_path,
    usecols=["Country", "Place.Name", "Latitude", "Longitude"],
    encoding="latin-1"
)

# Remove duplicate records
data = data.drop_duplicates()

# Change column name
data.rename(columns={"Place.Name": "Place_Name"}, inplace=True)

# Create a common column for cross join
data["K"] = 1

# Create all possible combinations of places
cross_data = pd.merge(data, data, on="K")

# Remove K column
```


Aaditya - ADS

```
cross_data.drop("K", axis=1, inplace=True)

# Rename columns
cross_data.rename(columns={
    "Country_x": "Country_from",
    "Country_y": "Country_to",
    "Place_Name_x": "Place_Name_from",
    "Place_Name_y": "Place_Name_to",
    "Latitude_x": "Latitude_from",
    "Latitude_y": "Latitude_to",
    "Longitude_x": "Longitude_from",
    "Longitude_y": "Longitude_to"
}, inplace=True)

# Calculate distance
cross_data["Distance_KM"] = cross_data.apply(
    lambda row: haversine(
        row["Longitude_from"],
        row["Latitude_from"],
        row["Longitude_to"],
        row["Latitude_to"]
    ),
    axis=1
)

# Convert kilometers to miles
cross_data["Distance_Miles"] = cross_data["Distance_KM"] * 0.621371

# Display dimensions
print("Rows and Columns:", cross_data.shape)

# Save the result
output_file =
"D:/practical-data-science/VKHCG/01-Vermeulen/01-Retrieve/01-EDS/02-Python/Retrieve_IP_Routi
ng.csv"

cross_data.to_csv(output_file, index=False, encoding="latin-1")

print("Done!")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4b_1.py
Rows and Columns: (9, 10)
Done!
```

Country_from	Place_Name_from	Latitude_from	Longitude_from	Country_to	Place_Name_to	Latitude_to	Longitude_to	Distance_KM	Distance_Miles
India	Mumbai	19.076	72.8777	India	Mumbai	19.076	72.8777	0	0
India	Mumbai	19.076	72.8777	India	Delhi	28.6139	77.209	1148.095	713.3929382
India	Mumbai	19.076	72.8777	United Kingdom	London	51.5074	-0.1278	7191.697	4468.711957
India	Delhi	28.6139	77.209	India	Mumbai	19.076	72.8777	1148.095	713.3929382
India	Delhi	28.6139	77.209	India	Delhi	28.6139	77.209	0	0
India	Delhi	28.6139	77.209	United Kingdom	London	51.5074	-0.1278	6711.247	4170.17426
United Kingdom	London	51.5074	-0.1278	India	Mumbai	19.076	72.8777	7191.697	4468.711957
United Kingdom	London	51.5074	-0.1278	India	Delhi	28.6139	77.209	6711.247	4170.17426
United Kingdom	London	51.5074	-0.1278	United Kingdom	London	51.5074	-0.1278	0	0

Aditya - ADS

4B2 Planning a schedule of jobs to be performed for the router network.

THEORY:
The program prepares a clean list of router locations by removing duplicate records and saving the cleaned data for further routing or scheduling operations.

CODE:
```
import pandas as pd

# Read CSV file
data = pd.read_csv(
    'D:/practical-data-science/VKHCG/01-Vermeulen/00-RawData/IP_DATA_CORE.csv',
    usecols=['Country', 'Place.Name', 'Latitude', 'Longitude'],
    encoding='latin-1'
)

# Rename column
data.rename(columns={'Place.Name': 'Place_Name'}, inplace=True)

print('Old rows:', len(data))

# Remove duplicates
data = data.drop_duplicates()

print('New rows:', len(data))
print('Columns:', len(data.columns))

# Save new CSV file
data.to_csv(

'D:/practical-data-science/VKHCG/01-Vermeulen/01-Retrieve/01-EDS/02-Python/Retrieve_Router_Location.csv',
    index=False
)

print('Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4b_2.py
Old rows: 6
New rows: 4
Columns: 4
Done!!
```

Country	Place_Name	Latitude	Longitude
India	Mumbai	19.076	72.8777
India	Delhi	28.6139	77.209
United Kingdom	London	51.5074	-0.1278
United States	New York	40.7128	-74.006

Aditya - ADS

4C Picking Content for Billboards

THEORY:
The program retrieves billboard location data, selects important columns, removes duplicate records and saves a cleaned dataset.

CODE:
```
import pandas as pd
# File details
input_file = 'DE_Billboard_Locations.csv'
output_file = 'Retrieve_DE_Billboard_Locations.csv'

base = 'D:/practical-data-science/VKHCG/'
company = '02-Krennwallner'

# Read CSV file
data = pd.read_csv(
    base + '/' + company + '/00-RawData/' + input_file,
    usecols=['Country', 'PlaceName', 'Latitude', 'Longitude']
)

# Rename column
data.rename(columns={'PlaceName': 'Place_Name'}, inplace=True)

# Display old number of rows
print('Rows from old set:', len(data))

# Remove duplicate rows
data = data.drop_duplicates()

# Display new size
print('Rows:', len(data))
print('Columns:', len(data.columns))

# Save cleaned data
data.to_csv(
    base + '/' + company + '/01-Retrieve/01-EDS/02-Python/' + output_file,
    index=False
)
print('Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4c.py
Rows from old set: 6
Rows after cleaning: 4
Columns: 4

Done!! Clean file saved.
```

Country	Place_Name	Latitude	Longitude
Germany	Berlin	52.52	13.405
Germany	Munich	48.1351	11.582
Germany	Hamburg	53.5511	9.9937
Germany	Frankfurt	50.1109	8.6821

Aditya - ADS

4D1 Planning the shipping rules for best-fit international logistics

THEORY:
This program retrieves shipping information and selects records having a particular shipping term, here **EXW**. It demonstrates **filtering data based on a condition**.

CODE:

```
import os
import pandas as pd
# Incoterm to search
term = 'EXW'
# File names
input_file = 'Incoterm_2010.csv'
output_file = 'Retrieve_Incoterm_EXW_RuleSet.csv'
base = 'D:/practical-data-science/VKHCG/'
company = '03-Hillman'
# Input and output paths
input_path = base + '/' + company + '/00-RawData/' + input_file
output_folder = base + '/' + company + '/01-Retrieve/01-EDS/02-Python'
# Create output folder
if not os.path.exists(output_folder):
    os.makedirs(output_folder)
# Read CSV
data = pd.read_csv(input_path)
# Select only EXW records
result = data[data['Shipping_Term'] == term]
# Display result
print('Rows:', len(result))
print('Columns:', len(result.columns))
print(result)
# Save result
output_path = output_folder + '/' + output_file
result.to_csv(output_path, index=False)
print('Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4d_1.py
Rows: 2
Columns: 4
   ID Shipping_Term       Description Cost_Bearer
0   1           EXW          Ex Works       Buyer
2   3           EXW  Ex Works Express       Buyer
Done!!
```

Aditya - ADS

4D2 Adopt New Shipping Containers

THEORY:

- ➢ This program generates data for containers, boxes and products based on their dimensions. It calculates their volumes and stores the generated information in CSV files.
- ➢ The program uses loops to generate many possible combinations of dimensions.
- ➢ DataFrame: A two-dimensional table-like data structure in Pandas.
- ➢ Why calculate volume? - To determine the space occupied by a container, box or product.

CODE:

```
import os
import pandas as pd
# Folder location
base = 'D:/practical-data-science/VKHCG/'
company = '03-Hillman'
folder = base + '/' + company + '/01-Retrieve/01-EDS/02-Python'
# Create folder if it does not exist
if not os.path.exists(folder):
    os.makedirs(folder)


# 1. CREATE CONTAINER DATA
containers = []
count = 0
for length in range(1, 21):
    for width in range(1, 10):
        for height in range(1, 6):
            count = count + 1
            volume = length * width * height
            containers.append([
                'Container',
                'C' + format(count, '06d'),
                length,
                width,
                height,
                volume
            ])
ContainerFrame = pd.DataFrame(containers, columns=[
    'ShipType',
    'UnitNumber',
    'Length',
    'Width',
    'Height',
    'ContainerVolume'
])
print('Container Rows:', len(ContainerFrame))
print('Container Columns:', len(ContainerFrame.columns))

ContainerFrame.to_csv(
    folder + '/Retrieve_Container.csv',
    index=False
)
# 2. CREATE BOX DATA
boxes = []
count = 0
for length in range(1, 21):
    for width in range(1, 21):
        for height in range(1, 21):
            for thickness in range(0, 6):
```


Aaditya - ADS

```
            box_volume = (length / 10) * (width / 10) * (height / 10)
            product_volume = (
                (length - thickness) / 10 *
                (width - thickness) / 10 *
                (height - thickness) / 10
            )
            if product_volume > 0:
                count = count + 1
                boxes.append([
                    'Box',
                    'B' + format(count, '06d'),
                    length / 10,
                    width / 10,
                    height / 10,
                    thickness / 5,
                    round(box_volume, 9),
                    round(product_volume, 9) ])
BoxFrame = pd.DataFrame(boxes, columns=[
    'ShipType',
    'UnitNumber',
    'Length',
    'Width',
    'Height',
    'Thickness',
    'BoxVolume',
    'ProductVolume'
])

print('Box Rows:', len(BoxFrame))
print('Box Columns:', len(BoxFrame.columns))
BoxFrame.to_csv(
    folder + '/Retrieve_Box.csv',
    index=False
)
# 3. CREATE PRODUCT DATA
products = []
count = 0
for length in range(1, 21):
    for width in range(1, 21):
        for height in range(1, 21):
            volume = (
                (length / 10) *
                (width / 10) *
                (height / 10)
            )
            count = count + 1
            products.append([
                'Product',
                'P' + format(count, '06d'),
                length / 10,
                width / 10,
                height / 10,
                round(volume, 9)
            ])

ProductFrame = pd.DataFrame(products, columns=[
    'ShipType',
    'UnitNumber',
```

Aaditya - ADS

```
    'Length',
    'Width',
    'Height',
    'ProductVolume'
])
print('Product Rows:', len(ProductFrame))
print('Product Columns:', len(ProductFrame.columns))
ProductFrame.to_csv(
    folder + '/Retrieve_Product.csv',
    index=False
)
print('Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4d_2.py
Container Rows: 900
Container Columns: 6
Box Rows: 34485
Box Columns: 8
Product Rows: 8000
Product Columns: 6
Done!!
```

4D3 Creating a delivery route

THEORY:

- This program prepares warehouse information and calculates the geographical distance between seller and buyer warehouses.
- It uses geopy and the geodesic distance method.
- Main steps
  - Read warehouse data.
  - Remove invalid coordinates.
  - Remove duplicate postcodes.
  - Sort warehouses.
  - Select sellers and buyers.
  - Calculate distance.
  - Save route information.
- Geodesic distance: The shortest distance between two points on the Earth's surface.
- Geopy: A Python library used for geographical calculations such as distances and coordinates.

CODE:

```
import os
import pandas as pd
from geopy.distance import geodesic
# File details
input_file = 'GB_Postcode_Warehouse.csv'
output_file = 'Retrieve_GB_Warehouse.csv'
base = 'D:/practical-data-science/VKHCG/'
company = '03-Hillman'
# Folder paths
input_path = base + '/' + company + '/00-RawData/' + input_file
output_folder = base + '/' + company + '/01-Retrieve/01-EDS/02-Python'
# Create output folder
if not os.path.exists(output_folder):
    os.makedirs(output_folder)
# Read warehouse data
warehouse = pd.read_csv(input_path)
# Remove warehouses with latitude or longitude = 0
warehouse = warehouse[
```


Aditya - ADS

```
    (warehouse['latitude'] != 0) &
    (warehouse['longitude'] != 0)
]
# Remove duplicate postcodes
warehouse = warehouse.drop_duplicates(
    subset='postcode'
)
# Sort by postcode
warehouse = warehouse.sort_values(
    by='postcode'
)
# Save cleaned warehouse data
warehouse.to_csv(
    output_folder + '/' + output_file,
    index=False
)
print('Warehouse data saved!')
# Use first 20 warehouses as sellers
sellers = warehouse.head(20)
# Use first 10,000 warehouses as buyers
buyers = warehouse.head(10000)
# Create routes for each seller
for i in range(len(sellers)):
    seller = sellers.iloc[i]
    print(
        'Run:', i,
        '======>',
        seller['postcode']
    )
    # Create a copy of buyer data
    routes = buyers.copy()
    # Seller information
    routes['Transaction'] = 'WH-to-WH'
    routes['Seller'] = 'WH-' + str(seller['postcode'])
    routes['Seller_Latitude'] = seller['latitude']
    routes['Seller_Longitude'] = seller['longitude']
    # Buyer information
    routes['Buyer'] = 'WH-' + routes['postcode'].astype(str)
    routes['Buyer_Latitude'] = routes['latitude'
    routes['Buyer_Longitude'] = routes['longitude']
    # Calculate distance
    routes['Distance'] = routes.apply(
        lambda row: round(
            geodesic(
                (seller['latitude'], seller['longitude']),
                (row['latitude'], row['longitude'])
            ).miles,
            6
        ),
        axis=1
    )
     routes = routes.drop(
        columns=['id', 'postcode', 'latitude', 'longitude']
    )
    output_name = (
        'Retrieve_Route_WH-' +
        str(seller['postcode']) +
        '_Route.csv'
```


Aditya - ADS

```
    )
    routes.to_csv(
        output_folder + '/' + output_name,
        index=False
    )
print('Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4d_3.py
Warehouse data saved!
Run: 0 ======> 110001.0
Run: 1 ======> 302001.0
Run: 2 ======> 400001.0
Run: 3 ======> 400020.0
Run: 4 ======> 400050.0
Done!!
```

4E Person Base data

THEORY:

- This program works with ZIP files containing name data. It extracts ZIP files, reads CSV files, combines them, renames the column, saves the result, and removes temporary extracted files.
- Important functions
  - **ZipFile()** → works with ZIP files.
  - **extractall()** → extracts files.
  - **os.walk()** → searches through folders.
  - **pd.concat()** → combines DataFrames.
  - **shutil.rmtree()** → removes a folder.

CODE:

```
import os
import zipfile
import shutil
import pandas as pd

base = "D:/practical-data-science/VKHCG/04-Clark"

zip_files = [
    "Data_female-names",
    "Data_male-names",
    "Data_last-names"
]

for name in zip_files:

    # ZIP and output paths
    zip_file = base + "/00-RawData/" + name + ".zip"
    folder = base + "/01-Retrieve/" + name
    output = base + "/01-Retrieve/Retrieve-" + name + ".csv"

    # Extract ZIP file
    with zipfile.ZipFile(zip_file, "r") as z:
        z.extractall(folder)

    # Read all CSV files
    files_data = []
```

Aaditya - ADS

```
    for root, folders, files in os.walk(folder):
        for file in files:
            if file.endswith(".csv"):
                data = pd.read_csv(
                    os.path.join(root, file),
                    header=None
                )
                files_data.append(data)

    # Combine CSV files
    result = pd.concat(files_data, ignore_index=True)

    # Rename column
    result.rename(
        columns={0: "NameValues"},
        inplace=True
    )

    # Save combined data
    result.to_csv(output, index=False)

    # Remove extracted files
    shutil.rmtree(folder)

    print("Processed:", name)

print("Done!")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical4>python Prac4e.py
Processed: Data_female-names
Processed: Data_male-names
Processed: Data_last-names
Done!
```

Aditya - ADS

PRACTICAL 5

AIM: Assessing Data

5A Drop the Columns Where All Elements Are Missing Values

THEORY:

- Data assessment checks the **quality and completeness of data**. Missing values are identified and handled according to the requirement.
- Data.dropna(axis=1, how='all') - This removes a column **only when every value in that column is missing**.
- axis=1 - Work with columns.
- Difference between all, thresh and any?
  - all: Remove when all values are missing.
  - any: Remove when at least one value is missing.
  - thresh=2: At least two non-null values must be present in the row.

CODE:

```
import pandas as pd
InputFile = 'Good-or-Bad.csv'
OutputFile = 'Good-or-Bad-01.csv'
Base = 'C:/VKHCG'
Company = '01-Vermeulen'
InputPath = Base + '/' + Company + '/00-RawData/' + InputFile
OutputPath = Base + '/' + Company + '/02-Assess/01-EDS/02-Python/' + OutputFile
print('Loading:', InputPath)
Data = pd.read_csv(InputPath)
print('## Raw Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
# Remove columns that contain only empty values
Data = Data.dropna(axis=1, how='all')
print('## Cleaned Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
Data.to_csv(OutputPath, index=False)
print('### Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical5>python Prac5a.py
Loading: Prac5a.csv
## Raw Data
   ID     Name  Age  Notes Status
0   1    Alice   25    NaN   Good
1   2      Bob   30    NaN    Bad
2   3  Charlie   35    NaN   Good
Rows: 3
Columns: 5
## Cleaned Data
   ID     Name  Age Status
0   1    Alice   25   Good
1   2      Bob   30    Bad
2   3  Charlie   35   Good
Rows: 3
Columns: 4
### Done!!
```

Aditya - ADS

5B Drop the Columns Where Any of the Elements Is Missing Values

CODE:

```
import pandas as pd
Base = 'C:/VKHCG'
Company = '01-Vermeulen'
InputFile = 'Good-or-Bad.csv'
OutputFile = 'Good-or-Bad-02.csv'
InputPath = Base + '/' + Company + '/00-RawData/' + InputFile
OutputPath = Base + '/' + Company + '/02-Assess/01-EDS/02-Python/' + OutputFile
print('Loading:', InputPath)
Data = pd.read_csv(InputPath)
print('## Raw Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
# Remove columns containing any empty value
Data = Data.dropna(axis=1, how='any')
print('## Cleaned Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
Data.to_csv(OutputPath, index=False)
print('### Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical5>python Prac5b.py
Loading: Prac5b.csv
## Raw Data
  ID,Name,Age,Notes,Status
0         1,Alice,25,,Good
1              2,Bob,,,Bad
2       3,Charlie,35,,Good
Rows: 3
Columns: 1
## Cleaned Data
  ID,Name,Age,Notes,Status
0         1,Alice,25,,Good
1              2,Bob,,,Bad
2       3,Charlie,35,,Good
Rows: 3
Columns: 1
### Done!!
```

5C Keep Only the Rows That Contain a Maximum of Two Missing Values

CODE:

```
import pandas as pd
Base = 'C:/VKHCG'
Company = '01-Vermeulen'
InputFile = 'Good-or-Bad.csv'
OutputFile = 'Good-or-Bad-03.csv'
InputPath = Base + '/' + Company + '/00-RawData/' + InputFile
OutputPath = Base + '/' + Company + '/02-Assess/01-EDS/02-Python/' + OutputFile
print('Loading:', InputPath)
Data = pd.read_csv(InputPath)
print('## Raw Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
# Keep rows having at least 2 non-empty values
Data = Data.dropna(thresh=2)
print('## Cleaned Data')
print(Data)
print('Rows:', Data.shape[0])
print('Columns:', Data.shape[1])
Data.to_csv(OutputPath, index=False)
print('### Done!!')
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical5>python Prac5c.py
Loading: Prac5c.csv
## Raw Data
   ID     Name   Age    Notes Status
0   1    Alice  25.0   Active   Good
1   2      NaN   NaN      NaN    NaN
2   3  Charlie   NaN      NaN    NaN
3   4      NaN   NaN      NaN    NaN
4   5      Eva  22.0  Pending   Good
Rows: 5
Columns: 5
## Cleaned Data
   ID     Name   Age    Notes Status
0   1    Alice  25.0   Active   Good
2   3  Charlie   NaN      NaN    NaN
4   5      Eva  22.0  Pending   Good
Rows: 3
Columns: 5
### Done!!
```

PRACTICAL 6

AIM: Processing Data -Build the time hub, links and satellites.

THEORY:

- This practical demonstrates a simple Data Vault-style structure using a time hub, links and satellites. The program creates time records and converts UTC time into different local time zones.
- Hub: The Hub-Time stores the main/common time information.
- Link: The Link connects the time information with a particular geographical/time-zone context.
- Satellite: The Satellite stores additional descriptive information such as the local time.
- UTC: Coordinated Universal Time, used as a common global time reference.
- The code converts UTC into:
    - Asia/Kolkata
    - Europe/London
    - America/New_York

CODE:

```
import pandas as pd
import sqlite3 as sq
from datetime import datetime, timedelta
from pytz import timezone

# Connect to database
conn = sq.connect('D:/Pictures/VKHCG/88-DV/datavault.db')

# ------------ TIME HUB ------------
dates = [datetime(2018, 1, 1) - timedelta(hours=i) for i in range(10)]

Hub = pd.DataFrame({
   'DateTimeKey': [d.strftime('%Y-%m-%d-%H-%M-%S') for d in dates],
   'DateTime': [d.strftime('%Y-%m-%d %H:%M:%S') for d in dates]
})

Hub.to_sql('Hub-Time', conn, if_exists='replace', index=False)

print("TIME HUB")
print(Hub)

# ---------- LINK & SATELLITE ----------
for zone in ['Asia/Kolkata', 'Europe/London', 'America/New_York']:

   Satellite = Hub.copy()

   Satellite['LocalTime'] = [
      d.replace(tzinfo=timezone('UTC')).astimezone(timezone(zone))
      .strftime('%Y-%m-%d %H:%M:%S')
      for d in dates
   ]

   Satellite[['DateTimeKey']].to_sql(
      'Link-' + zone.replace('/', '-'),
      conn, if_exists='replace', index=False
   )
```

Aditya - ADS

```
    Satellite.to_sql(
        'Satellite-' + zone.replace('/', '-'),
        conn, if_exists='replace', index=False
    )

    print("\n", zone)
    print(Satellite)

conn.close()

print("\nDONE!")
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical6>python Prac6.py
TIME HUB
           DateTimeKey             DateTime
0  2018-01-01-00-00-00  2018-01-01 00:00:00
1  2017-12-31-23-00-00  2017-12-31 23:00:00
2  2017-12-31-22-00-00  2017-12-31 22:00:00
3  2017-12-31-21-00-00  2017-12-31 21:00:00
4  2017-12-31-20-00-00  2017-12-31 20:00:00
5  2017-12-31-19-00-00  2017-12-31 19:00:00
6  2017-12-31-18-00-00  2017-12-31 18:00:00
7  2017-12-31-17-00-00  2017-12-31 17:00:00
8  2017-12-31-16-00-00  2017-12-31 16:00:00
9  2017-12-31-15-00-00  2017-12-31 15:00:00

 America/New_York
           DateTimeKey             DateTime           LocalTime
0  2018-01-01-00-00-00  2018-01-01 00:00:00  2017-12-3119:00:00
1  2017-12-31-23-00-00  2017-12-31 23:00:00  2017-12-3118:00:00
2  2017-12-31-22-00-00  2017-12-31 22:00:00  2017-12-3117:00:00
3  2017-12-31-21-00-00  2017-12-31 21:00:00  2017-12-3116:00:00
4  2017-12-31-20-00-00  2017-12-31 20:00:00  2017-12-3115:00:00
5  2017-12-31-19-00-00  2017-12-31 19:00:00  2017-12-3114:00:00
6  2017-12-31-18-00-00  2017-12-31 18:00:00  2017-12-3113:00:00
7  2017-12-31-17-00-00  2017-12-31 17:00:00  2017-12-3112:00:00
8  2017-12-31-16-00-00  2017-12-31 16:00:00  2017-12-3111:00:00
9  2017-12-31-15-00-00  2017-12-31 15:00:00  2017-12-3110:00:00

DONE!
```

Aa ADS

PRACTICAL 7

AIM: Transforming Data - To illustrate the consolidation process

THEORY:

- Data transformation changes data from one form or format into another so that it can be stored and used consistently.
- This practical converts UTC time into local time and stores the transformed information in both a Data Vault database and a Data Warehouse database.
- Main steps
  - Connect to two SQLite databases.
  - Create UTC date/time.
  - Convert it to Asia/Kolkata time.
  - Create a consolidated DataFrame.
  - Store it in the Data Vault.
  - Store it in the Data Warehouse.
  - Create person information using the converted time.

CODE:

```
import pandas as pd
import sqlite3
from datetime import datetime
from pytz import timezone

# Connect databases
db1 = sqlite3.connect("D:/Pictures/VKHCG/88-DV/datavault.db")
db2 = sqlite3.connect("D:/Pictures/VKHCG/88-DV/datawarehouse.db")

# Original UTC data
utc = datetime(2001,6, 24, 10, 15).replace(tzinfo=timezone("UTC"))

# Transform time zone
zone = "Asia/Kolkata"
local = utc.astimezone(timezone(zone))
# Consolidate time data
time = pd.DataFrame([{
   "DateTime": utc.strftime("%Y-%m-%d %H:%M:%S"),
   "Zone": zone,
   "LocalTime": local.strftime("%Y-%m-%d %H:%M:%S")
}])
# Store transformed data
time.to_sql("Hub-Time", db1, if_exists="replace", index=False)
time.to_sql("Dim-Time", db2, if_exists="replace", index=False)
# Person data
person = pd.DataFrame([{
   "Name": "Sophia Dcruz",
   "BirthTime": local.strftime("%Y-%m-%d %H:%M:%S")
}])
person.to_sql("Hub-Person", db1, if_exists="replace", index=False)
person.to_sql("Dim-Person", db2, if_exists="replace", index=False)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical7>python Prac7.py
UTC Time: 2005-04-14 10:15:00+00:00
Local Time: 2005-04-14 15:45:00+05:30

Consolidated Data:
            Name            BirthTime
0  Aaditya Mishra  2005-04-14 15:45:00
```

Aaditya - ADS

PRACTICAL 8

AIM: Organizing Data

THEORY:

- This practical demonstrates different ways of organizing or presenting data from a data warehouse into a data mart.
- SQL queries are used to select and filter the required BMI information.

8A Horizontal Style

THEORY:

- The program selects BMI records where:
  - Height > 1.5
  - Indicator = 1
- The selected data is stored in a separate datamart table called Dim-BMI-Horizontal.
- Horizontal organization keeps multiple attributes/columns together for selected records.

CODE:

```
import pandas as pd
import sqlite3

# Connect to databases
conn1 = sqlite3.connect("datawarehouse.db")
conn2 = sqlite3.connect("datamart.db")

# Read complete BMI data
data = pd.read_sql_query("SELECT * FROM [Dim-BMI]", conn1)

# Filter BMI data
query = """
SELECT PersonID, Height, Weight, bmi, Indicator
FROM [Dim-BMI]
WHERE Height > 1.5
AND Indicator = 1
ORDER BY Height, Weight
"""

filtered = pd.read_sql_query(query, conn1)

# Set PersonID as index
filtered.set_index("PersonID",  inplace=True)

# Store filtered data in datamart
filtered.to_sql(
    "Dim-BMI-Horizontal",
    conn2,
    if_exists="replace"
)

# Read stored data
result = pd.read_sql_query(
    "SELECT * FROM [Dim-BMI-Horizontal]",
    conn2
)

# Display information
print("Full Data Set")
```

Aaditya - ADS

```
print("Rows:", data.shape[0])
print("Columns:", data.shape[1])

print("\nFiltered Data Set")
print("Rows:", result.shape[0])
print("Columns:", result.shape[1])

print("\nFiltered BMI Data:")
print(result)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical8\Practical8a>python Prac8a.py
Full Data Set
Rows: 5
Columns: 5

Filtered Data Set
Rows: 3
Columns: 5

Filtered BMI Data:
   PersonID  Height  Weight    bmi  Indicator
0       101    1.75    70.0  22.86          1
1       103    1.82    85.0  25.66          1
2       105    1.90    92.0  25.48          1
```

8B Vertical Syle

THEORY:

- ➢ Vertical organization focuses on selected attributes/columns rather than keeping the complete original table.
- ➢ Horizontal selection mainly focuses on selecting records/rows, while vertical selection focuses on selecting attributes/columns.

CODE:

```
import pandas as pd
import sqlite3

# Connect to databases
conn1 = sqlite3.connect("datawarehouse.db")
conn2 = sqlite3.connect("datamart.db")

# Read complete BMI data
data = pd.read_sql_query(
    "SELECT * FROM [Dim-BMI]",
    conn1
)

# Select required columns
query = """
SELECT Height, Weight, Indicator
FROM [Dim-BMI]
"""

filtered = pd.read_sql_query(query, conn1)

# Set Indicator as index
filtered.set_index("Indicator", inplace=True)
```


Aaditya - ADS

```
# Store as vertical BMI table
filtered.to_sql(
    "Dim-BMI-Vertical",
    conn2,
    if_exists="replace"
)

# Read the stored table
result = pd.read_sql_query(
    "SELECT * FROM [Dim-BMI-Vertical]",
    conn2
)

# Display information
print("Full Data Set")
print("Rows:", data.shape[0])
print("Columns:", data.shape[1])

print("\nVertical Data Set")
print("Rows:", result.shape[0])
print("Columns:", result.shape[1])

print("\nVertical BMI Data:")
print(result)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical8\Practical8b>python Prac8b.py
Full Data Set
Rows: 5
Columns: 5

Vertical Data Set
Rows: 5
Columns: 3

Vertical BMI Data:
   Indicator  Height  Weight
0          1    1.75    70.0
1          1    1.40    45.0
2          1    1.82    85.0
3          0    1.60    58.0
4          1    1.90    92.0
```

8C Island Style

THEORY:

An island-style dataset is a small, independent subset of the larger dataset created for a particular purpose.

CODE:

```
import pandas as pd
import sqlite3

# Connect to databases
conn1 = sqlite3.connect("datawarehouse.db")
conn2 = sqlite3.connect("datamart.db")

# Read complete BMI data
data = pd.read_sql_query(
    "SELECT * FROM [Dim-BMI]",
```

```
    conn1
)

# Filter BMI data
query = """
SELECT Height, Weight, Indicator
FROM [Dim-BMI]
WHERE Indicator > 2
ORDER BY Height, Weight
"""

filtered = pd.read_sql_query(query, conn1)

# Set Indicator as index
filtered.set_index("Indicator", inplace=True)

# Store filtered data
filtered.to_sql(
    "Dim-BMI-Island",
    conn2,
    if_exists="replace"
)

# Read the stored table
result = pd.read_sql_query(
    "SELECT * FROM [Dim-BMI-Island]",
    conn2
)

# Display information
print("Full Data Set")
print("Rows:", data.shape[0])
print("Columns:", data.shape[1])

print("\nIsland Data Set")
print("Rows:", result.shape[0])
print("Columns:", result.shape[1])

print("\nBMI Island Data:")
print(result)
```

OUTPUT:

```
C:\Users\ADITYA\Downloads\Ads_Practical\Practical8\Practical8c>python Prac8c.py
Full Data Set
Rows: 5
Columns: 5

Island Data Set
Rows: 0
Columns: 3

BMI Island Data:
Empty DataFrame
Columns: [Indicator, Height, Weight]
Index: []
```

Aditya - ADS

PRACTICAL 9

AIM: Case Study on Sales Details

THEORY:

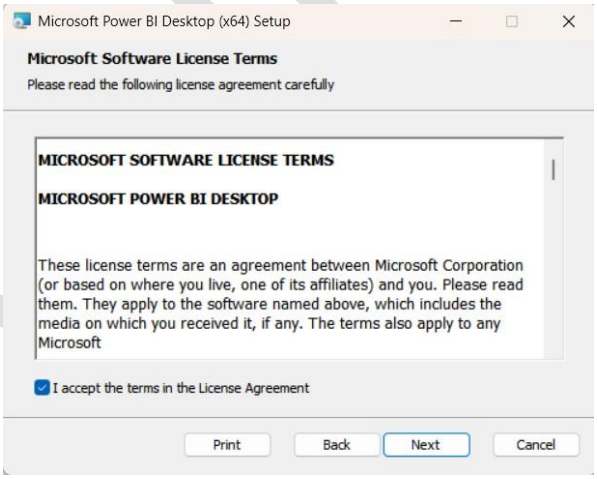
- Power BI is a data visualization and business intelligence tool. It converts raw data into charts, tables, cards and interactive reports so that patterns and trends can be understood easily. The uploaded practical uses a sales case study.
- Important Visualizations
  - Bar/Column Chart: Compare categories
  - Pie Chart: Show proportions
  - Line Chart: Show trends over time
  - Table: Display detailed records
  - Card: Display one important number
  - Scatter Plot: Show relationship between numerical variables
  - Slicer: Filter the report interactively
- These visualizations and field arrangements are specified in the uploaded practical

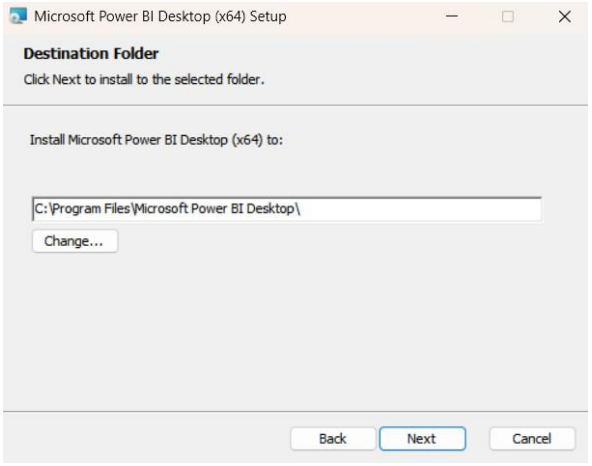
STEPS FOR INSTALLATIONS

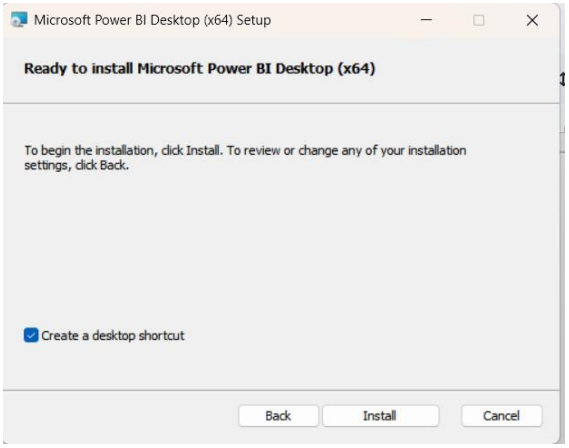
1. Go to the official Microsoft Power BI website.
2. Click Download Free.
3. Sign in with your Microsoft account if required.
4. Download the Power BI Desktop installer.
5. Open the downloaded installer.
6. Accept the terms and click Install.
7. After installation, open Power BI Desktop from the Start Menu.









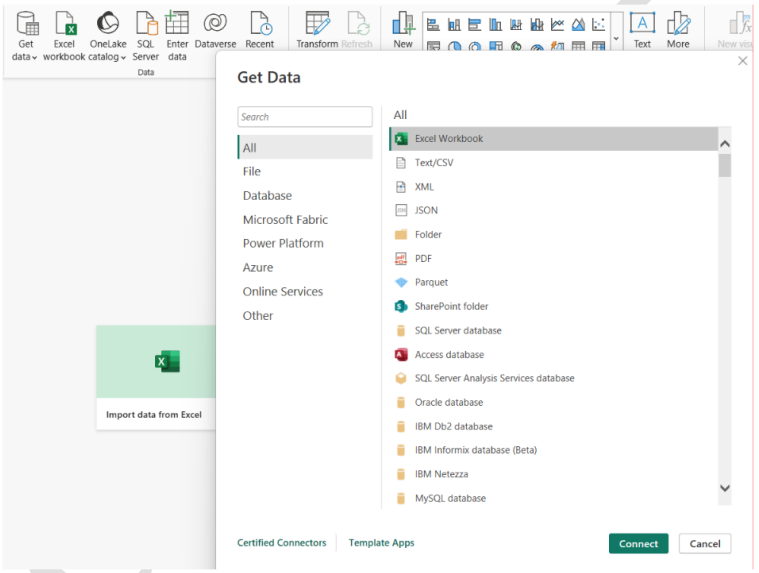


STEPS TO PERFORM:

1. Open Power BI

1. Open **Microsoft Power BI Desktop**.
2. Click on **Home → Get Data**.
3. Select the required data source, such as:
   - Excel
   - CSV
   - Text file
   - Database

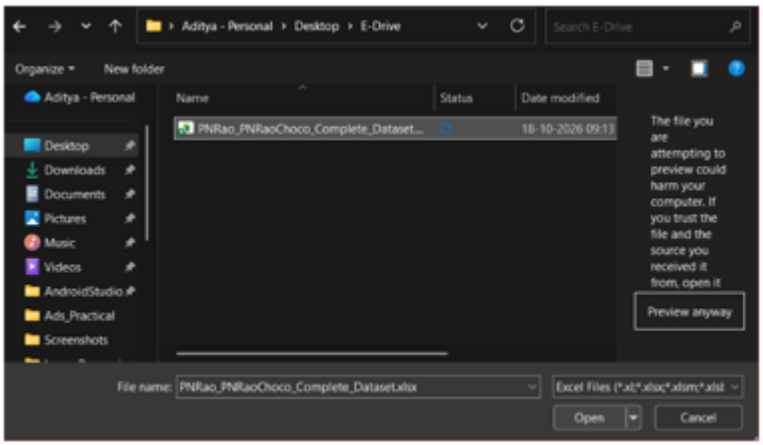
Output:

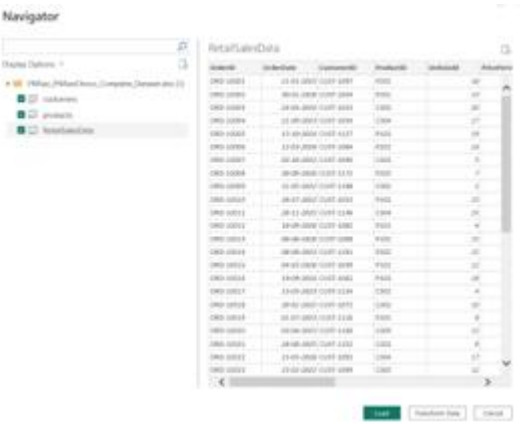


2. Import the Dataset

1. Select the required dataset.
2. Click **Open**.
3. The **Navigator** window will appear.
4. Select the required table/sheet.
5. Click **Load** to import the data.

Output:

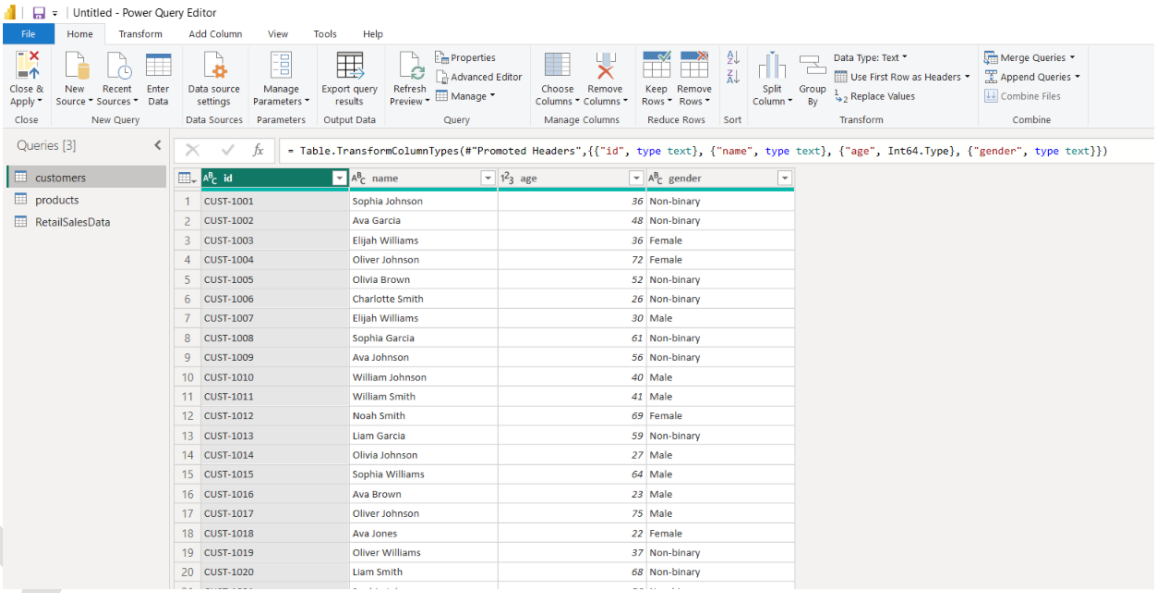




3. Check and Clean the Data

1. Go to **Home → Transform Data**.
2. Power Query Editor will open.
3. Check the column names and data types.
4. Remove unnecessary columns if required.
5. Remove duplicate or blank records if necessary.
6. Correct incorrect data types.
7. Click **Close & Apply**.

Output:



4. Create a Visualization

1. Select the **Report** view.
2. Choose a visualization from the **Visualizations** pane.
3. Drag the required fields into the appropriate areas. For example:

Visualization	Fields
Bar Chart	Category → X-axis, Sales → Y-axis
Pie Chart	Category → Legend, Sales → Values

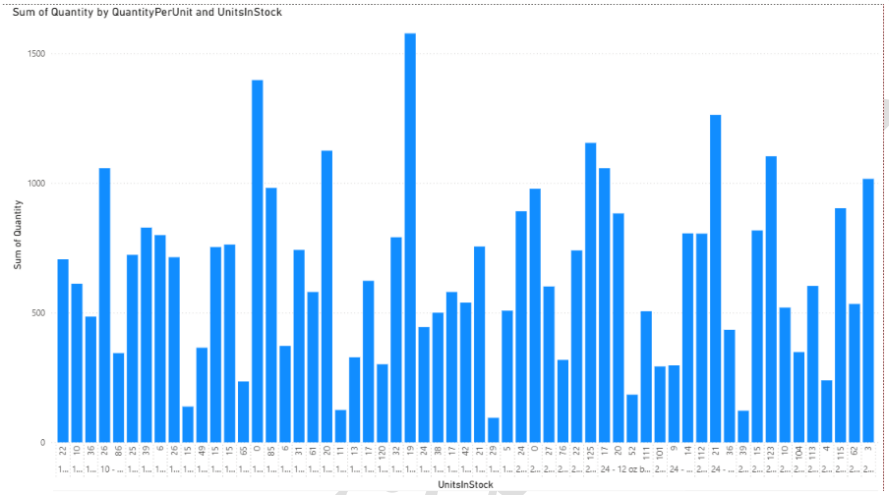
Line Chart	Date → X-axis, Sales → Y-axis

Table	Select required fields
Card	Select a numerical field
Scatter Plot	Numerical field → X/Y-axis

5. Create a Bar Chart

1. Select **Clustered Column Chart** or **Bar Chart**.
2. Drag a categorical field such as **Category/Company** to the Axis.
3. Drag a numerical field such as **Sales/Revenue** to Values.
4. Power BI automatically generates the chart.

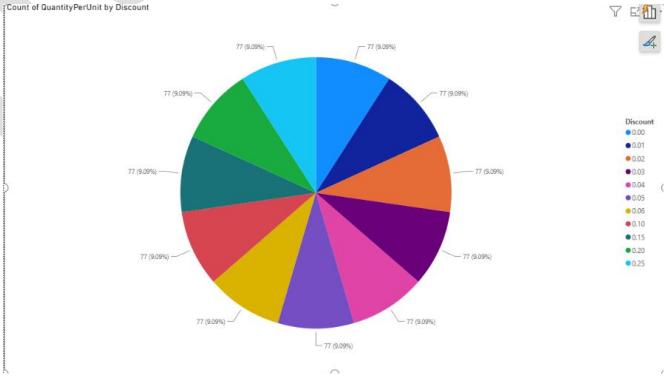
Output:



6. Create a Pie Chart

1. Select **Pie Chart**.
2. Drag the categorical field to **Legend**.
3. Drag the numerical field to **Values**.
4. The pie chart displays the proportion of each category.

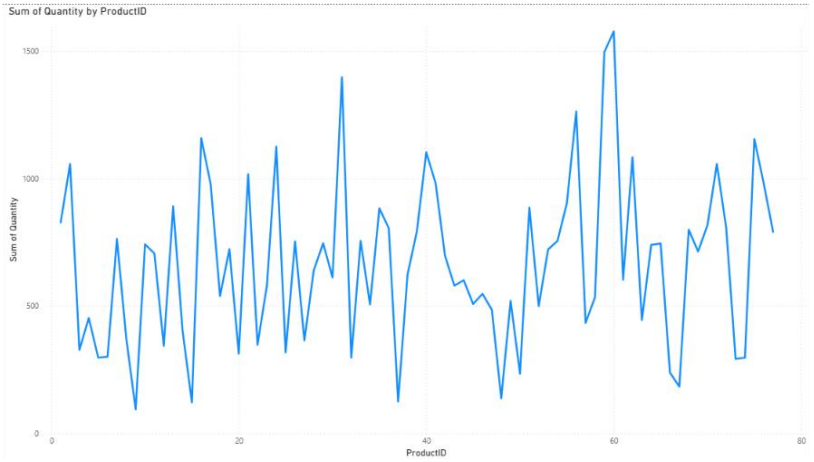
Output:



7. Create a Line Chart

1. Select **Line Chart**.
2. Drag **Date/Year/Month** to the X-axis.
3. Drag the numerical field such as **Sales** to the Y-axis.
4. The chart shows the trend over time.

Output:



8. Create a Table

1. Select **Table** from the Visualizations pane.
2. Drag the required fields into the table.
3. The selected data will be displayed in tabular form.

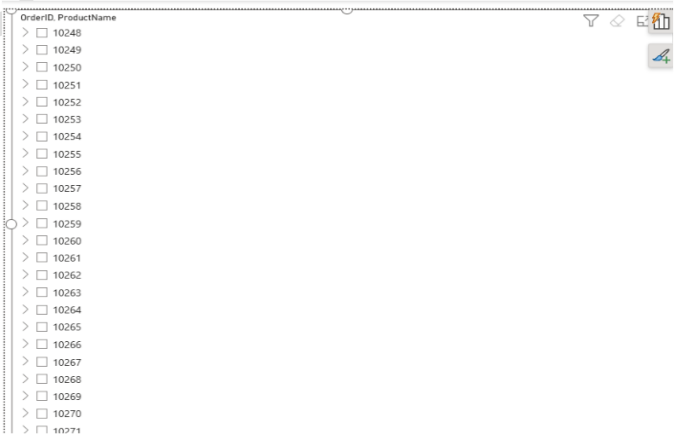
Output:

ProductID	ProductName	QuantityPerUnit	Sum of Discount	Sum of UnitPrice
1	Chai	10 boxes x 20 bags	2.95	651.60
2	Chang	24 - 12 oz bottles	4.50	786.60
3	Aniseed Syrup	12 - 550 ml bottles	0.20	114.00
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	1.50	413.60
5	Chef Anton's Gumbo Mix	36 boxes	0.75	196.10
6	Grandma's Boysenberry Spread	12 - 8 oz jars	0.37	290.00
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	0.80	846.00
8	Northwoods Cranberry Sauce	12 - 12 oz jars	0.85	504.00
9	Mishi Kobe Niku	18 - 500 g pkgs.	0.50	465.60
10	Ikura	12 - 200 ml jars	1.45	979.60
11	Queso Cabrales	1 kg pkg.	2.15	744.80
12	Queso Manchego La Pastora	10 - 500 g pkgs.	0.90	516.80
13	Konbu	2 kg box	1.70	230.40
14	Tofu	40 - 100 g pkgs.	1.48	469.65
15	Genen Shouyu	24 - 250 ml bottles	0.10	86.80
16	Pavlova	32 - 500 g boxes	2.88	704.20
17	Alice Mutton	20 - 1 kg tins	2.30	1,349.40
18	Carnarvon Tigers	16 kg pkg.	2.15	1,612.50
19	Teatime Chocolate Biscuits	10 boxes x 12 pieces	2.05	315.70
20	Sir Rodney's Marmalade	30 gift boxes	0.74	1,215.00
21	Sir Rodney's Scones	24 pkgs. x 4 pieces	2.20	366.00
22	Gustaf's Knäckebröd	24 - 500 g pkgs.	0.30	285.60
23	Tunnbröd	12 - 250 g pkgs.	0.75	167.40
24	Guaraná Fantástica	12 - 355 ml cans	2.90	216.00
25	NuNuCa Nuß-Nougat-Creme	20 - 450 g glasses	1.55	235.20
Total			**121.04**	**56,500.91**

9. Add a Slicer

1. Select **Slicer**.
2. Drag a field such as **Category, Company, Year, or Region** into the slicer.
3. Select different values from the slicer.
4. Other visualizations on the page will automatically filter according to the selection.

Output:

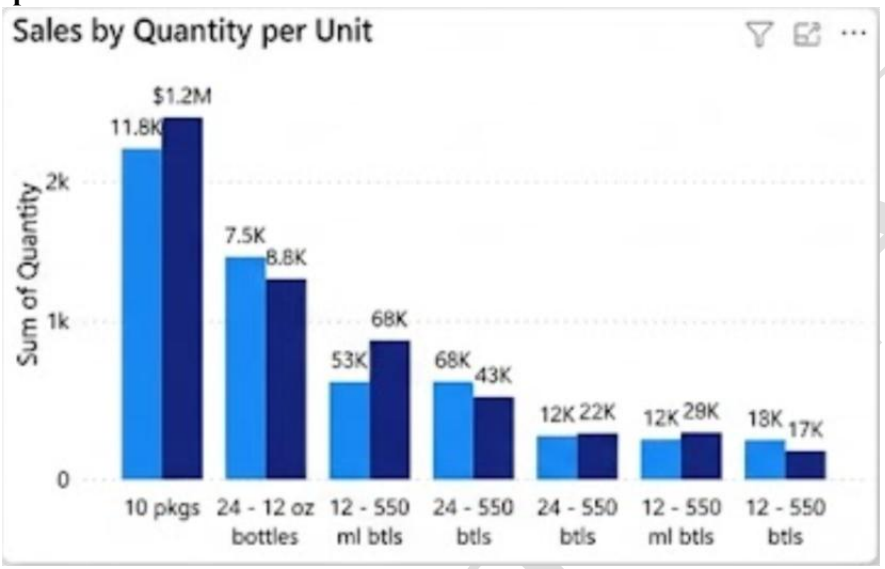


Aaij's ADS

10. Format the Visualizations

1. Select a chart.
2. Open the **Format** pane.
3. Modify:
    - Chart title
    - Axis titles
    - Data labels
    - Legend
    - Background
    - Font size
4. Arrange the visualizations properly on the report page.

Output:



11. Create the Dashboard/Report

Combine different visualizations on one report page, for example:

- **Card** → Total Sales
- **Bar Chart** → Sales by Category
- **Pie Chart** → Category-wise Percentage
- **Line Chart** → Sales Trend**Table** → Detailed Records
- **Slicer** → Filter by Year/Category

Aadiya ADS

Output:



12. Analyze the Results

Use the visualizations to identify:

- Highest and lowest values
- Trends over time
- Category-wise differences
- Distribution of data
- Relationships between variables
- Important patterns or outliers

13. Save the Report

1. Click **File → Save As**.
2. Enter a suitable name.
3. Save the file with the **.pbix** extension.

Output:

